Big Data Assignment 2
Simple Search Engine using Hadoop MapReduce, Cassandra, and Spark

Makar Egorov

March 24, 2026

1 Introduction

The purpose of this assignment is to implement a simple distributed search engine for plain text documents using Hadoop MapReduce, Cassandra, and Spark. The system is required to support document preparation, index construction, persistent storage of index structures, and ranked query answering based on BM25. The implementation is deployed in Docker containers and is launched with a single `docker compose up` command.

The final system consists of three main stages. First, documents are extracted from a Wikipedia parquet file and transformed into plain text files in the required naming format. Second, the documents are indexed in Hadoop using MapReduce pipelines, and the resulting index structures are stored in Cassandra. Third, a Spark application reads the stored index, computes BM25 scores for query terms, and returns the top relevant documents.

2 Methodology

2.1 System Overview

The project implements a simple distributed search engine composed of the following components:

- **PySpark** for data preparation and ranked retrieval.
- **Hadoop MapReduce** for offline batch indexing.
- **Cassandra** for storing index structures and corpus statistics.
- **Docker Compose** for orchestrating the full environment.

The system runs in three containers:

- `hadoop-master`
- `hadoop-worker`
- `cassandra`

The master node automatically executes the script `app/app.sh`, which performs the complete pipeline:

1. waits for Cassandra to become available,
2. prepares data from the parquet file,
3. uploads the generated documents to HDFS,
4. builds the index using Hadoop Streaming MapReduce jobs,
5. stores the resulting index into Cassandra,
6. runs demo search queries using Spark on YARN.

2.2 Data Preparation

The input dataset is a Wikipedia parquet file containing article identifiers, titles, and text content. In this project, one parquet shard was used:

```
app/data_source/a.parquet
```

The preparation step is implemented in `prepare_data.py` using PySpark. The script reads the parquet file, selects the fields `id`, `title`, and `text`, removes invalid or empty rows, and extracts a fixed number of documents. For the demonstration required in the report, the number of documents was set to 100.

For each document, a plain text file is created using the required naming format:

```
<doc_id>_<doc_title>.txt
```

The title is normalized to a filesystem-safe ASCII representation to avoid issues with special characters during HDFS upload.

In addition, the preparation step creates a one-partition text file for the indexing pipeline. Each line has the following format:

```
<doc_id>\t<doc_title>\t<doc_text>
```

After preparation, the outputs are uploaded into HDFS:

- `/data` for the individual text documents,
- `/input/data` for the one-partition input file used by the indexer.

2.3 Index Design

The search engine stores the following information:

- document metadata,
- postings lists,
- vocabulary document frequencies,
- corpus statistics required for BM25.

The final index in Cassandra contains four tables:

- `documents(doc_id, title, doc_len)`
- `terms(term, df)`
- `postings(term, doc_id, tf)`
- `corpus_stats(stat_key, stat_value)`

This schema was selected because BM25 requires:

- term frequency $tf(t, d)$,
- document frequency $df(t)$,
- document length $dl(d)$,
- total number of documents N,
- average document length $avgdl$.

2.4 MapReduce Indexing Pipelines

The indexing phase is implemented as two Hadoop Streaming MapReduce jobs.

2.4.1 Pipeline 1

The first pipeline processes each prepared input line and generates:

- `DOC` records containing document id, title, and document length,
- `POSTING` records containing term, document id, and term frequency.

The mapper tokenizes the document text using lowercase alphanumeric tokens and computes term frequencies inside each document. The reducer forwards the intermediate output into a single consolidated stage output.

2.4.2 Pipeline 2

The second pipeline processes the output of stage 1 and computes:

- `TERM` records with document frequency df,
- `STAT` records for N and $avgdl$.

The result is stored in HDFS under:

- `/indexer/stage1`
- `/indexer/stage2`

2.5 Loading the Index into Cassandra

After index generation in HDFS, the script `store_index.sh` exports the stage outputs from HDFS into temporary local TSV files. Then `app.py` connects to Cassandra and performs the following operations:

1. creates keyspace `search_engine` if it does not already exist,
2. creates all required tables,
3. truncates the tables to avoid duplicates on repeated runs,
4. inserts document records, postings, vocabulary statistics, and corpus statistics.

The Cassandra keyspace uses `SimpleStrategy` with replication factor 1, which is sufficient for the assignment environment.

2.6 Ranking Model

The ranker uses BM25. For each query term t and document d, the score is computed as:

$$BM25(t,d) = \log\left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1\right) \cdot \frac{tf(t,d)(k_1+1)}{tf(t,d) + k_1\left((1-b) + b \cdot \frac{dl(d)}{avgdl}\right)}$$

where:

- N is the total number of documents,
- $df(t)$ is the number of documents containing term t,
- $tf(t,d)$ is the term frequency in document d,
- $dl(d)$ is the document length,
- $avgdl$ is the average document length,
- $k_1 = 1.2$,
- $b = 0.75$.

For multi-term queries, the final document score is the sum of BM25 scores over all matching query terms.

2.7 Query Processing with Spark

The query processor is implemented in `query.py`. It runs as a Spark application on YARN and performs the following steps:

1. reads the query text,
2. tokenizes and normalizes it,
3. connects to Cassandra,
4. retrieves *df* values for query terms,
5. retrieves postings for matching terms,
6. retrieves document metadata,
7. computes BM25 scores using Spark RDD transformations,
8. returns the top 10 documents ranked by score.

The application includes connection retry logic for Cassandra in order to make the automated pipeline more stable.

3 Demonstration

3.1 How to Run the Repository

The repository can be started with a single command from the project root:

```
docker compose up
```

This command creates and starts three containers:

- Hadoop master node,
- Hadoop worker node,
- Cassandra server.

The master container automatically executes `app/app.sh`, which runs the complete pipeline from data preparation to query execution.

3.2 Successful Pipeline Execution

Figure 1 shows the successful end-to-end execution of the full repository. The screenshot includes the final message `Pipeline finished successfully`, which confirms that all stages completed:

- document preparation,
- HDFS upload,
- MapReduce indexing,
- Cassandra storage,
- Spark-based query answering.

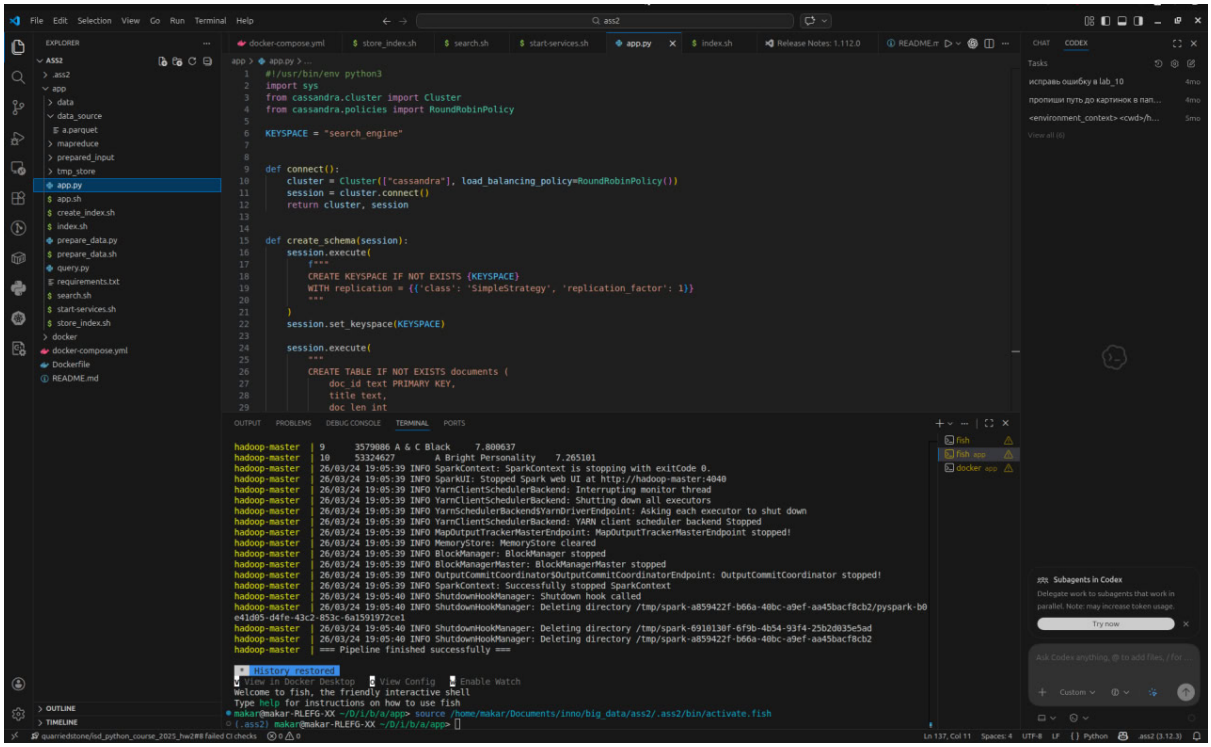

Figure 1: Successful end-to-end pipeline execution after running `docker compose up`.

3.3 Successful Indexing of 100 Documents

Figure 2 demonstrates that 1000 documents were prepared and uploaded into HDFS. The screenshot shows the HDFS count for `/data` as well as the prepared input file inside `/input/data`. This confirms that the required document subset was created successfully and is ready for indexing.

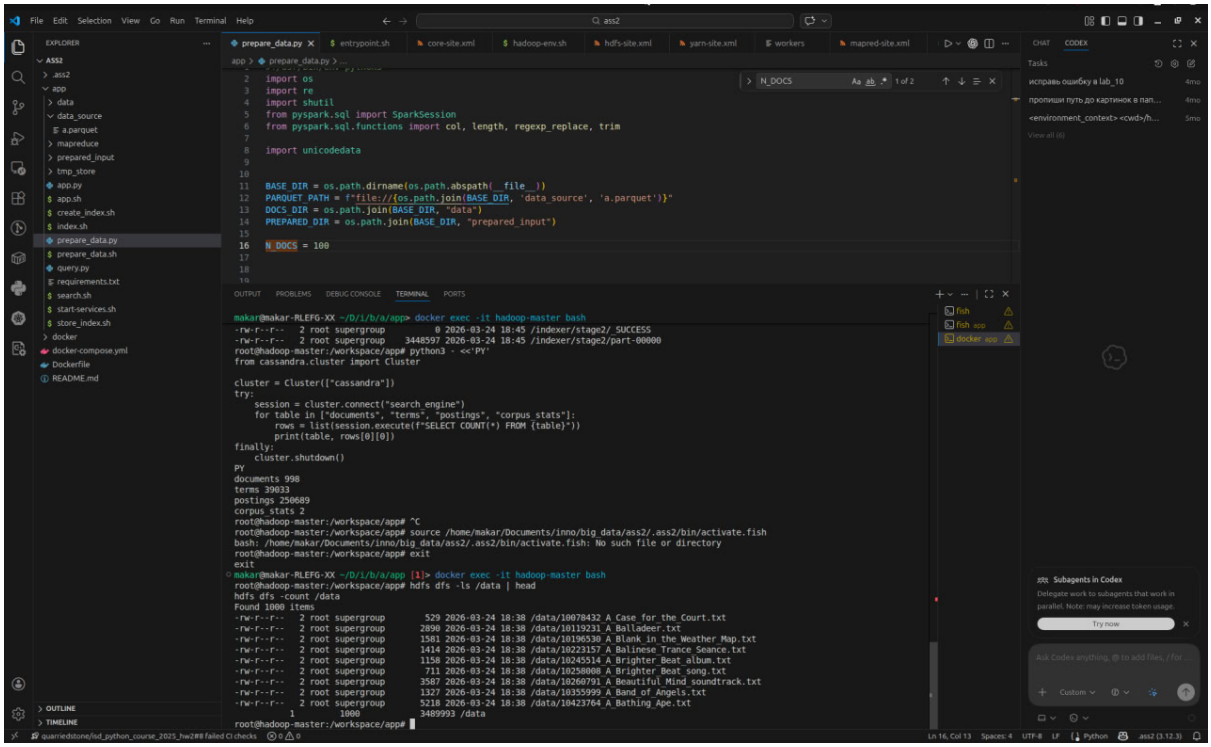

Figure 2: Successful preparation and upload of 100 documents into HDFS.

3.4 Successful MapReduce Index Construction

Figure 3 shows the outputs of the two indexing stages in HDFS. Stage 1 contains document records and posting entries, while Stage 2 contains vocabulary statistics and corpus statistics. This demonstrates that the MapReduce pipelines produced the required index structures for later ranking.

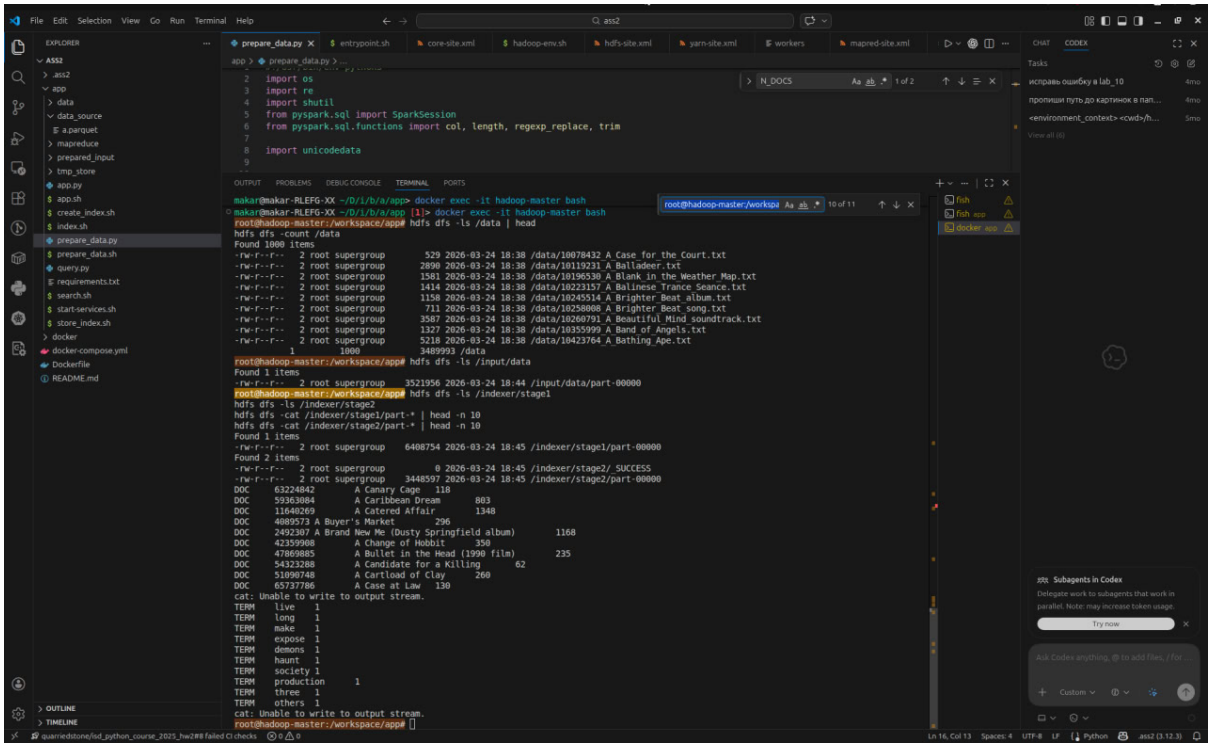

Figure 3: Successful MapReduce indexing output stored in `/indexer/stage1` and `/indexer/stage2`.

3.5 Successful Cassandra Loading

Figure 4 shows that the generated index was loaded into Cassandra. The screenshot includes the counts from the four main tables:

- `documents`
- `terms`
- `postings`
- `corpus_stats`

  The values confirm that the index structures were successfully persisted.

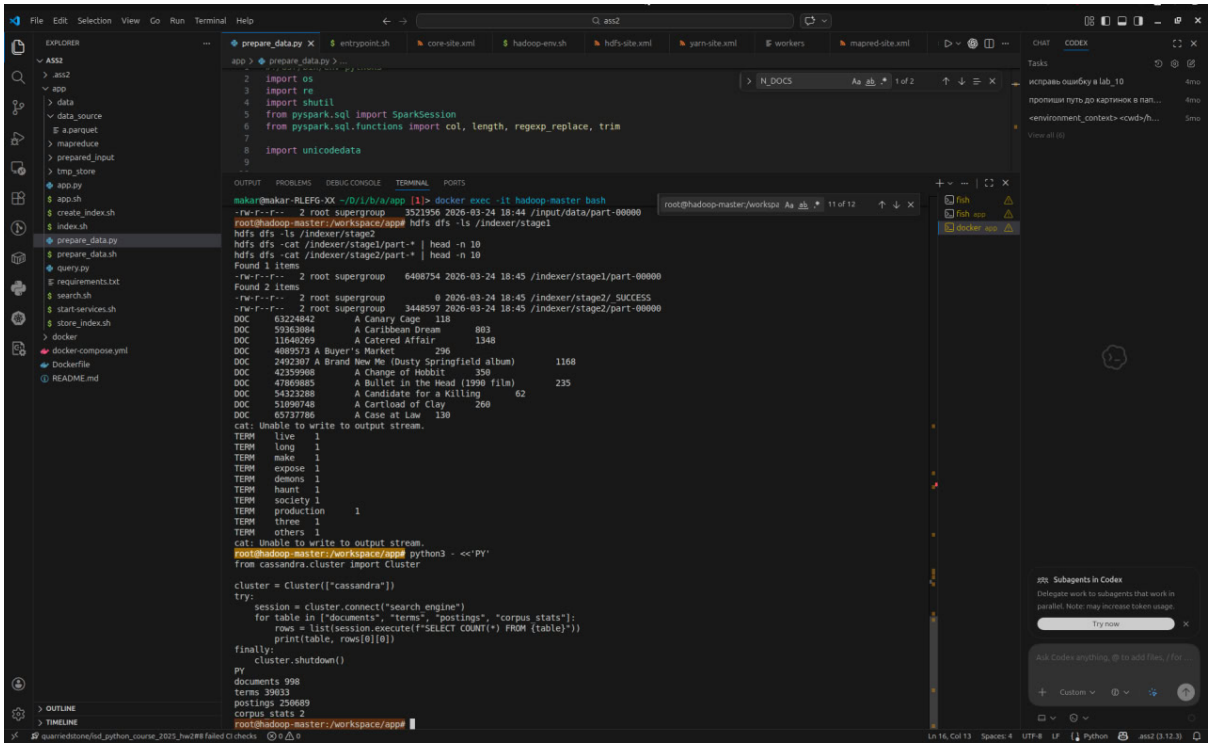

Figure 4: Successful storage of the index structures in Cassandra.

3.6 Successful Query Execution

The search engine was tested with multiple example queries. Figures 5, ??, and ?? show the top-10 ranked documents returned by BM25 for the queries `film`, `album`, and `publishing`.

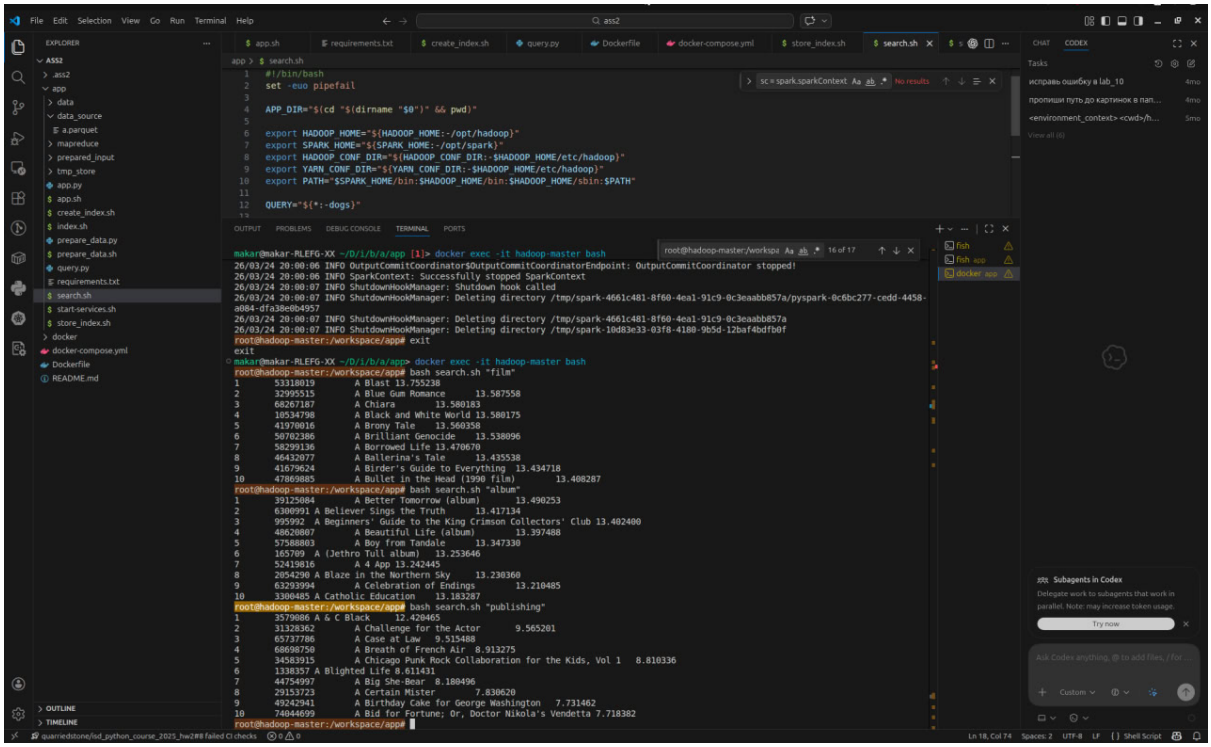

Figure 5: Top-10 results for the query `film`.

3.7 Explanations of the Retrieved Results

The returned results are generally consistent with the meaning of the query terms.

For the query `film`, the result list contains film-related pages and entertainment entities. One example is `A Bullet in the Head (1990 film)`, which clearly matches the query topic. Some other retrieved pages are not exact film titles but still belong to nearby cultural or entertainment topics. This happens because BM25 is based on lexical statistics and retrieves documents where the query term is frequent or important.

For the query `album`, the returned documents include pages explicitly related to music albums and music works. This indicates that the system is correctly matching terms associated with album titles or album-related descriptions.

For the query `publishing`, the results include pages related to publishing organizations, book-related topics, and cultural production. One example is `A & C Black`, which is a publishing company and therefore highly relevant.

3.8 Reflections on the Findings

The implemented system demonstrates a complete distributed retrieval pipeline and successfully integrates Hadoop, Cassandra, and Spark. The results show that BM25 provides meaningful ranking based on token frequency, document frequency, and document length normalization.

At the same time, the output also highlights some limitations of the current implementation:

- the tokenizer is simple and only uses lowercase alphanumeric terms,
- there is no stopword removal,
- there is no stemming or lemmatization,
- ranking is purely lexical and does not use semantic similarity.

These limitations explain why some returned documents are only partially related to the query topic rather than being exact matches. Nevertheless, for a simple BM25-based search engine, the system performs reasonably well and satisfies the assignment requirements.

4 Conclusion

This project successfully implements a simple search engine using Hadoop MapReduce, Cassandra, and Spark. The system prepares text documents from a parquet source, stores them in HDFS, builds an inverted index using MapReduce, loads the index into Cassandra, and answers user queries with BM25 ranking executed by Spark on YARN.

The final repository runs automatically through `docker compose up`, which makes the project reproducible and easy to demonstrate. The screenshots confirm that the implementation supports successful indexing, persistent storage of the index, and ranked retrieval on multiple example queries.

The system can be improved further with more advanced preprocessing and ranking features, but even in its current form it demonstrates the core components of a distributed search engine and fulfills the main goals of the assignment.